

Model Selection

Empirical risk minimization (ERM)

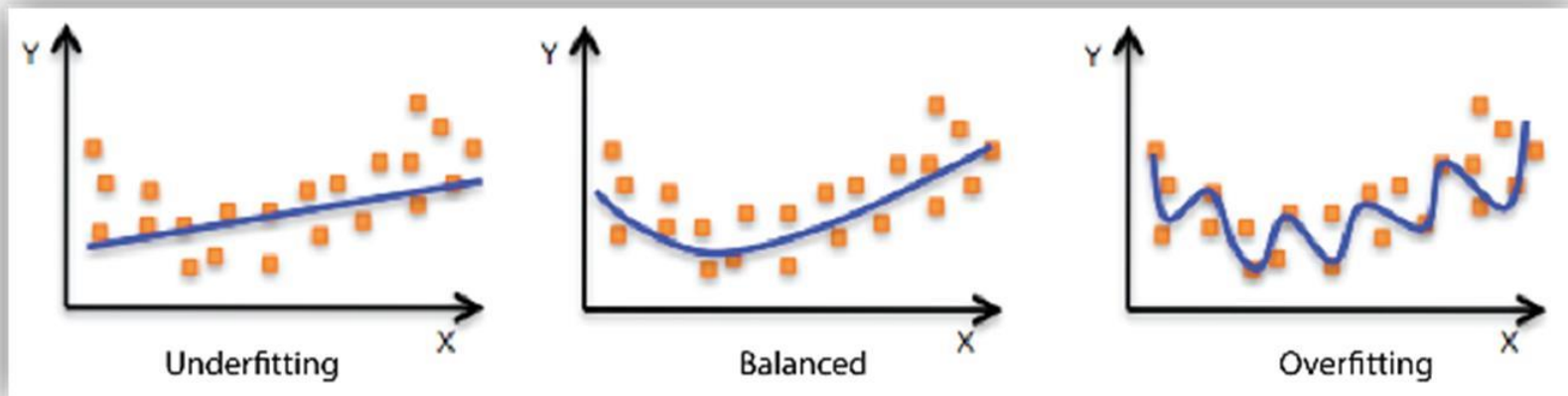
Structural risk minimization (SRM)

Regularization

cross-validation

What is Model complexity?

Complexity	Problem
Too low	Underfitting
Too high	Overfitting
Just right	Good generalization



Empirical Risk Minimization (ERM)

- ERM means **choosing** the model that minimizes error on training data.
- It is the principle of choosing a **model (hypothesis)** that minimizes the **empirical risk**, i.e., the **average loss over the training data**.

$$\hat{h} = \arg \min_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n L(h(x_i), y_i)$$

Where,

$\mathcal{H}$: hypothesis space

L : loss function

(x_i, y_i) : training samples

$h \in \mathcal{H}$

h : a model (e.g., linear regression, polynomial, etc.)

$\mathcal{H}$: set of all possible models (**hypothesis space**)

$\arg \min$ -choose the value that minimizes

Find the model h that gives the **smallest average error on the training data**

- **Key idea** - Fit the model as well as **possible to the observed data.**
- **Limitation:**
 - ERM **ONLY cares** about training data
 - Does **NOT consider complexity** that's why, ERM can lead to **overfitting**
 - especially if the hypothesis space is very large or complex.

Structural Risk Minimization (SRM)

- SRM don't just minimize error also control model complexity.

$$\text{Total Risk} = \text{Empirical Risk} + \text{Complexity}$$

- While **ERM** minimizes the empirical risk, it does not consider the complexity of the hypothesis.
- Structural Risk Minimization (SRM) addresses this by adding a regularization term to **penalize** overly complex models.
- SRM helps achieve a balance between underfitting and overfitting, making it a more robust approach

Comparison

Aspect	ERM	SRM
Focus	Training error	Training error + model complexity
Risk minimized	Empirical risk	Structural risk
Overfitting control	No	Yes
Generalization	Not guaranteed	Better generalization

Regularization

- In order to **create less complex model** when you have a large number of features in your dataset, some of the Regularization techniques used to address **over-fitting and feature selection** are:
 - L1 Regularization
 - L2 Regularization

Regularization

- Regularization is a technique to prevent overfitting by adding a **penalty for model complexity**.

$$\text{Loss} = \text{Empirical Risk} + \lambda \cdot \text{Penalty}$$

- λ - controls **strength of regularization**
- Higher $\lambda \rightarrow$ simpler model
- Regularization encourages **simpler models** and enhances generalization.

Why Regularization is used ?

- **Problem:**
 - Complex model → **overfits**
 - Fits noise in training data
- **Goal:**
 - Balance:
 - Fit (low error)
 - Simplicity (low complexity)

L1 Norm (Manhattan Norm)

- The L1 norm of a vector is the **sum of the absolute values** of its components.

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

For $x = [3, -4]$:

$$\|x\|_1 = |3| + |-4| = 3 + 4 = 7$$

L2 Norm (Euclidean Norm)

- The L2 norm is the **square root of the sum of squares** of components.

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

Example:

For $x = [3, -4]$:

$$\|x\|_2 = \sqrt{3^2 + (-4)^2} = \sqrt{9 + 16} = 5$$

L1 and L2

- L1 and L2 regularisation owes its name to L1 and L2 norm of a vector $\mathbf{w}$ respectively. Here's a primer on norms:
- L1 Norm

$$\|\mathbf{w}\|_1 = |w_1| + |w_2| + \dots + |w_N|$$

- L2 Norm

$$\|\mathbf{w}\|_2 = \left(|w_1|^2 + |w_2|^2 + \dots + |w_N|^2\right)^{\frac{1}{2}}$$

L1 Regularization (Lasso)

$$\text{Loss} = \text{Error} + \lambda \sum |w_i|$$

- Penalizes **absolute values of weights**
- Some weights become **exactly zero**

Result

- Performs **feature selection**
- Produces **sparse models**

L2 Regularization (Ridge)

$$\text{Loss} = \text{Error} + \lambda \sum w_i^2$$

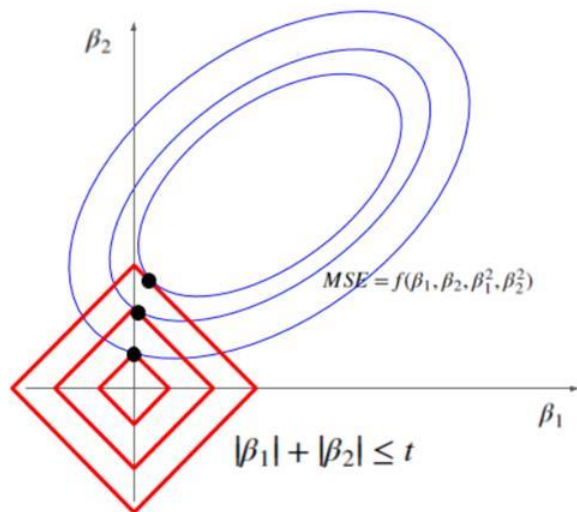
- Penalizes **squared weights**
- Weights become **small but not zero**

Result

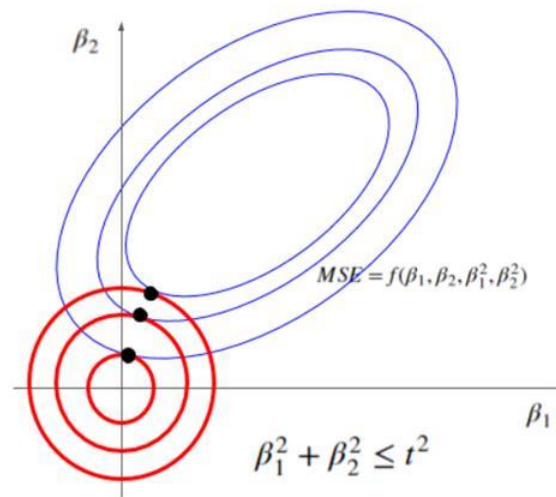
- Keeps all features
- Produces **smooth models**

Geometric Intuition

Shrink t , and check how the corresponding β_1 β_2 behave



L1 regularization



L2 regularization

L1 (Lasso)	L2 (Ridge)
• Shape: Diamond • Has corners • Solution hits axes • Some coefficients = 0	• Shape: Circle • Smooth boundary • Coefficients $\neq 0$, just smaller

When to Use?

- **Use L1 (Lasso) when:**
 - Many irrelevant features
 - Need feature selection
- **Use L2 (Ridge) when:**
 - All features are important
 - Want stable model

Revisiting Regression to understand regularization

Errors in Linear Regression

$$Loss = Error(y, \hat{y})$$

Loss function with no regularisation

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N |w_i|$$

Loss function with L1 regularisation

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N w_i^2$$

Loss function with L2 regularisation

Ridge Regression

- A regression model that uses **L1 regularization** technique is called **Lasso Regression** and model which uses **L2** is called **Ridge Regression**.
- The key difference between these two is the **penalty term**.
- Ridge regression adds “**squared magnitude**” of coefficient as penalty term to the loss function. Here the highlighted part represents L2 regularization element.

Ridge Regression – Cost Function

$$\sum_{i=1}^n (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

- Here, if lambda is zero then we can imagine we get back OLS.
- However, if lambda is very large then it will add too much weight and it will lead to **under-fitting**.
- Having said that it's important **how lambda is chosen**.
This technique works very well to avoid over-fitting issue.

Lasso Regression – Cost Function

- Lasso Regression (Least Absolute Shrinkage and Selection Operator) adds “absolute value of magnitude” of coefficient as **penalty term** to the loss function.

$$\sum_{i=1}^n (Y_i - \sum_{j=1}^p X_{ij}\beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- Again, if lambda is zero then we will get back OLS whereas **very large value** will make coefficients **zero** hence it will **under-fit**.

Comparing

- The key difference between these techniques is that **Lasso shrinks the less important feature's coefficient to zero thus, removing some feature altogether.**
- So, this works well for feature selection in case we have a **huge number of features.**

Elastic Net

- Elastic net is a popular type of regularized linear regression that combines two popular penalties, specifically **the L1 and L2 penalty functions**.
- **a hyperparameter “alpha”** is provided to assign **how much weight** is given to each of the L1 and L2 penalties.
- **Alpha is a value between 0 and 1** and is used to weight the contribution of the L1 penalty and one minus the alpha value is used to weight the L2 penalty.

$\text{elastic_net_penalty} = \alpha \cdot \text{L1 penalty} + (1 - \alpha) \cdot \text{L2 penalty}$

- Standard Form:

$$\lambda \left(\alpha \sum |w_i| + (1 - \alpha) \sum w_i^2 \right)$$

- **L1 penalty** = $\sum |w_i|$
- **L2 penalty** = $\sum w_i^2$
- **α (alpha)** = balance between L1 and L2 ✓
- **λ (lambda)** = overall strength of regularization

Dataset: Boston Housing

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	<u>"crim"</u>	<u>"zn"</u>	<u>"indus"</u>	<u>"chas"</u>	<u>"nox"</u>	<u>"rm"</u>	<u>"age"</u>	<u>"dis"</u>	<u>"rad"</u>	<u>"tax"</u>	<u>"ptratio"</u>	<u>"b"</u>	<u>"lstat"</u>	<u>"medv"</u>
2	0.00632	18	2.31	"0"	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
3	0.02731	0	7.07	"0"	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
4	0.02729	0	7.07	"0"	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
5	0.03237	0	2.18	"0"	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
6	0.06905	0	2.18	"0"	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
7	0.02985	0	2.18	"0"	0.458	6.43	58.7	6.0622	3	222	18.7	394.12	5.21	28.7
8	0.08829	12.5	7.87	"0"	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9
9	0.14455	12.5	7.87	"0"	0.524	6.172	96.1	5.9505	5	311	15.2	396.9	19.15	27.1
10	0.21124	12.5	7.87	"0"	0.524	5.631	100	6.0821	5	311	15.2	386.63	29.93	16.5
11	0.17004	12.5	7.87	"0"	0.524	6.004	85.9	6.5921	5	311	15.2	386.71	17.1	18.9
12	0.22489	12.5	7.87	"0"	0.524	6.377	94.3	6.3467	5	311	15.2	392.52	20.45	15
13	0.11747	12.5	7.87	"0"	0.524	6.009	82.9	6.2267	5	311	15.2	396.9	13.27	18.9
14	0.09378	12.5	7.87	"0"	0.524	5.889	39	5.4509	5	311	15.2	390.5	15.71	21.7
15	0.62976	0	8.14	"0"	0.538	5.949	61.8	4.7075	4	307	21	396.9	8.26	20.4
16	0.63796	0	8.14	"0"	0.538	6.096	84.5	4.4619	4	307	21	380.02	10.26	18.2
17	0.62739	0	8.14	"0"	0.538	5.834	56.5	4.4986	4	307	21	395.62	8.47	19.9
18	1.05393	0	8.14	"0"	0.538	5.935	29.3	4.4986	4	307	21	386.85	6.58	23.1
19	0.7842	0	8.14	"0"	0.538	5.99	81.7	4.2579	4	307	21	386.75	14.67	17.5
20	0.80271	0	8.14	"0"	0.538	5.456	36.6	3.7965	4	307	21	288.99	11.69	20.2

Read libraries and dataset

```
import numpy as np
import pandas as pd
import math
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
#import training dataset
train_df = pd.read_csv('BostonHousing.csv')
```

```
X = train_df.drop('medv', axis=1) #input
y = train_df['medv'] #output

X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=0, test_size=0.3)
```

Basic Linear Model

```
lr = LinearRegression()
lr.fit(X_train, y_train)

print('Training Acc:', lr.score(X_train, y_train)*100)
print('Test Acc:', lr.score(X_test, y_test)*100)

y_pred = lr.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = math.sqrt(mse)

print('RMSE:', rmse)
```

Training Acc: 76.4545102694255

Test Acc: 67.33825506400194

RMSE: 5.214975145375405

Polynomial Model

```
steps = [  
    ('scalar', StandardScaler()),  
    ('poly', PolynomialFeatures(degree=2)),  
    ('model', LinearRegression())  
]  
  
pipeline = Pipeline(steps)  
  
pipeline.fit(X_train, y_train)  
  
print('Training Acc:', pipeline.score(X_train, y_train)*100)  
print('Test Acc:', pipeline.score(X_test, y_test)*100)
```

Training Acc: 95.16134299280634

Test Acc: 64.62815294725782

Ridge Regression

```
steps = [  
    ('scalar', StandardScaler()),  
    ('poly', PolynomialFeatures(degree=2)),  
    ('model', Ridge(alpha=10, fit_intercept=True))  
]  
  
ridge = Pipeline(steps)  
ridge.fit(X_train, y_train)  
  
print('Training Acc:', ridge.score(X_train, y_train)*100)  
print('Test Acc:', ridge.score(X_test, y_test)*100)
```

Training Acc: 93.7200975636591

Test Acc: 79.09830005263463

Lasso Regression

```
steps = [  
    ('scalar', StandardScaler()),  
    ('poly', PolynomialFeatures(degree=2)),  
    ('model', Lasso(alpha=0.3, fit_intercept=True))  
]  
  
lasso = Pipeline(steps)  
  
lasso.fit(X_train, y_train)  
  
print('Training Acc:', lasso.score(X_train, y_train)*100)  
print('Test Acc:', lasso.score(X_test, y_test)*100)
```

Training Acc: 87.19391604372422

Test Acc: 76.41692818439569

ElasticNet Regression

```
steps = [  
    ('scalar', StandardScaler()),  
    ('poly', PolynomialFeatures(degree=2)),  
    ('model', ElasticNet(alpha=0.1, fit_intercept=True))  
]  
  
eln = Pipeline(steps)  
  
eln.fit(X_train, y_train)  
  
print('Training Acc:', eln.score(X_train, y_train)*100)  
print('Test Acc:', eln.score(X_test, y_test)*100)
```

Training Acc: 91.89478837576667

Test Acc: 81.01310659764083

Result Comparison

Basic Linear Model	Polynomial Model	Ridge Regression (L2 Regularization)	Lasso Regression (L1 Regularization)
Train Acc: 76.45 Test Acc: 67.34 RMSE: 5.21	Train Acc: 95.16 (very high) Test Acc: 64.63 (low)	Train Acc: 93.72 Test Acc: 79.10 (best)	Train Acc: 87.19 Test Acc: 76.42
- Moderate training performance - Slight drop on test data - This indicates underfitting (high bias)	- Excellent training performance - Poor test performance - This is classic overfitting (high variance)	- Slightly lower training accuracy than polynomial - Much better test accuracy - This shows good generalization	Lower training accuracy than Ridge - Still strong test performance

What is Cross-Validation?

- Cross-validation is a technique used to **evaluate model performance** on unseen data by splitting data into **multiple** training and validation sets.

Why Do We Need It?

- **Problem:**

- Single train-test split can give **biased results**
- Model may perform well just by chance

- **Solution:**

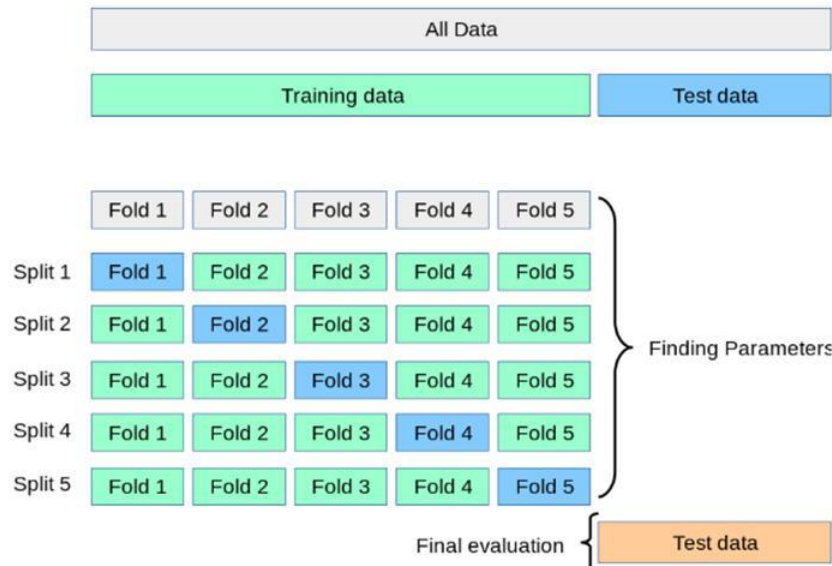
- Cross-validation gives a **more reliable estimate of model performance**

3. How It Works (k-Fold Cross-Validation)

- **Steps:**
 - Split data into **k equal parts (folds)**
 - Train on **k-1 folds**
 - Test on **remaining 1 fold**
 - Repeat **k times**
 - Take **average performance**

k-Fold Cross Validation

- k-Fold cross-validation is a technique used to evaluate a machine learning model by dividing the dataset into k equal parts (folds), training the model on $k-1$ folds, and testing it on the remaining fold, repeated k times.
- k-fold cross-validation where $k = n$ (number of data points)



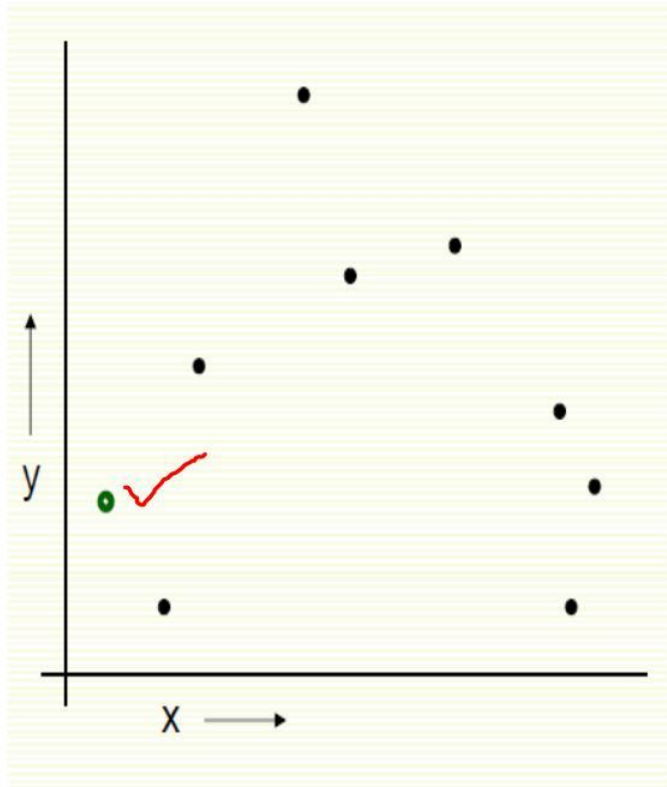
In **k-fold CV**:

- Data is split into **k parts**
- Each fold is used once as test set

Leave-One-Out (LOOCV)

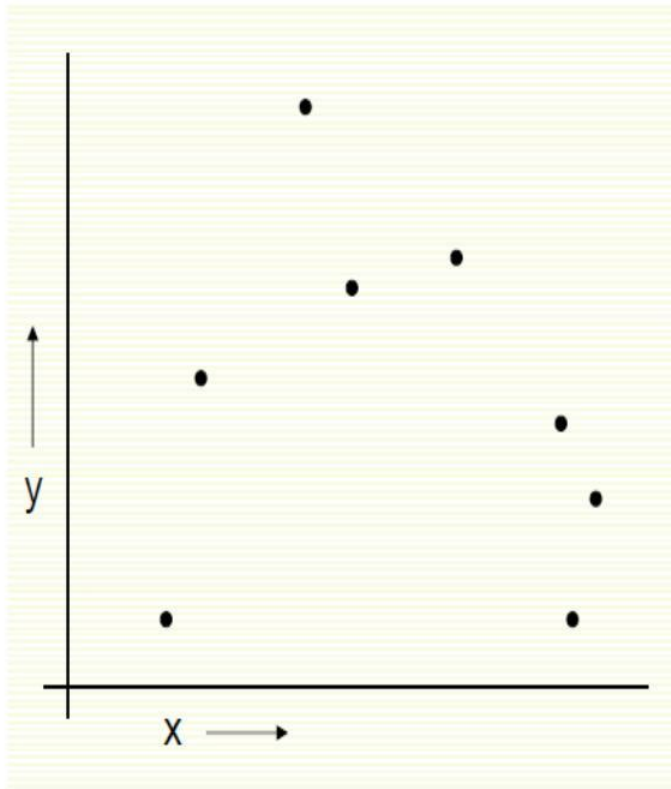
- In **LOOCV**:
 - Each fold has **exactly 1 data point**
 - So number of folds = number of samples
 - Each sample is a test set once
 - Very expensive

LOOCV (Leave-one-out Cross Validation)



- For $k=1$ to R
 1. Let $(\mathbf{x}^k, \mathbf{y}^k)$ be the $\mathbf{k}$ example

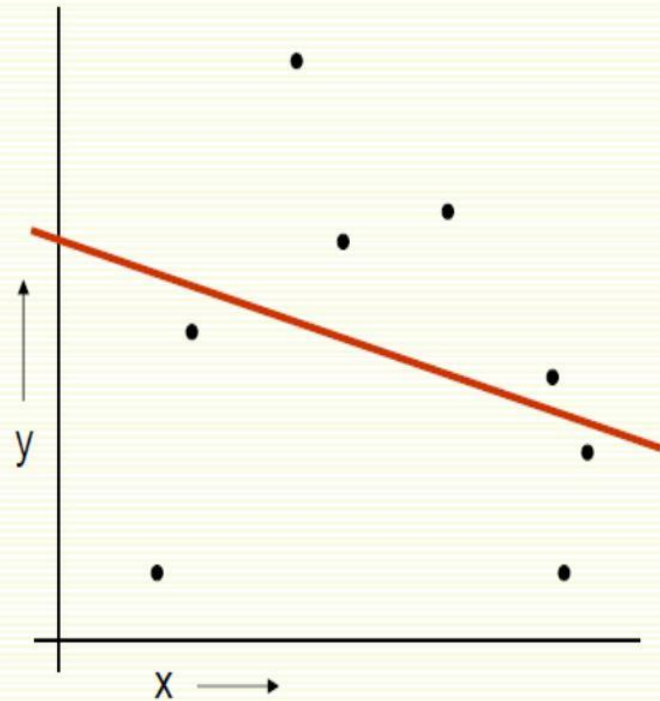
LOOCV (Leave-one-out Cross Validation)



For $k=1$ to n

1. Let $(\mathbf{x}^k, \mathbf{y}^k)$ be the k th example
2. Temporarily remove $(\mathbf{x}^k, \mathbf{y}^k)$ from the dataset

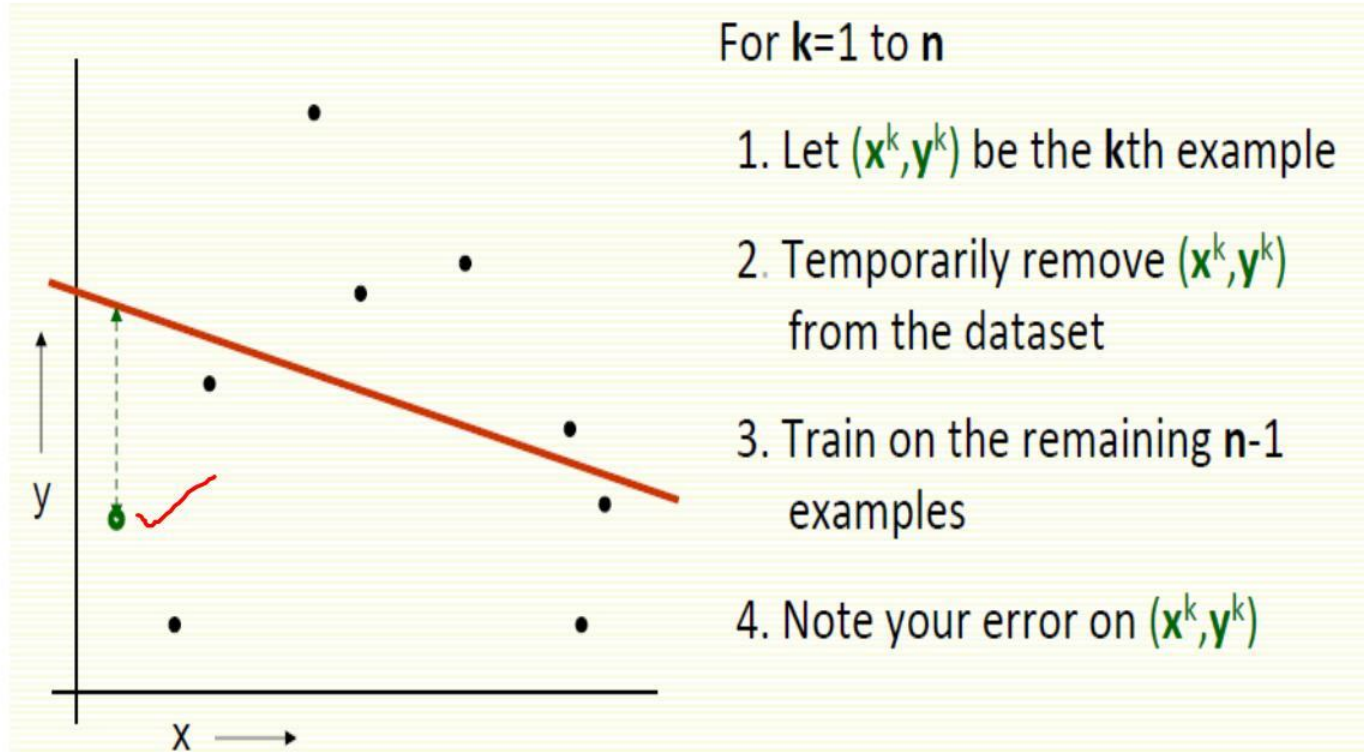
LOOCV (Leave-one-out Cross Validation)



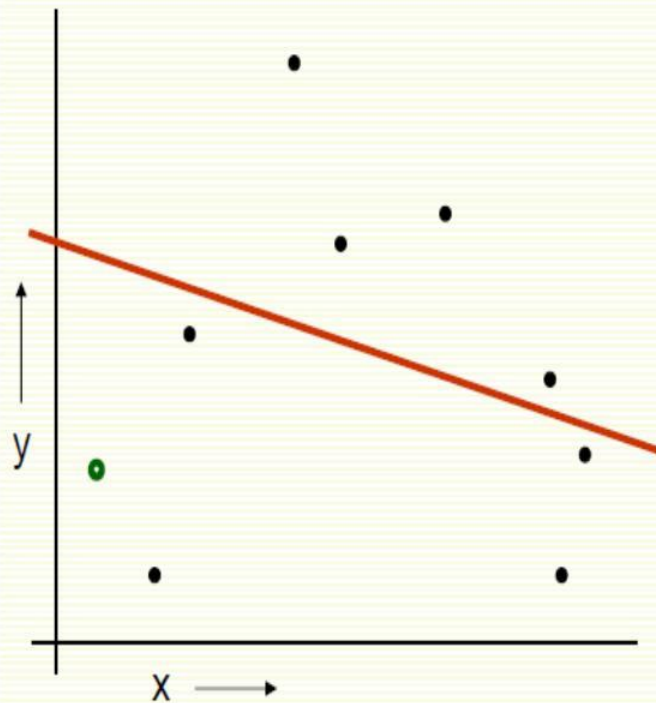
For $k=1$ to n

1. Let $(\mathbf{x}^k, \mathbf{y}^k)$ be the k th example
2. Temporarily remove $(\mathbf{x}^k, \mathbf{y}^k)$ from the dataset
3. Train on the remaining $n-1$ examples

LOOCV (Leave-one-out Cross Validation)



LOOCV (Leave-one-out Cross Validation)

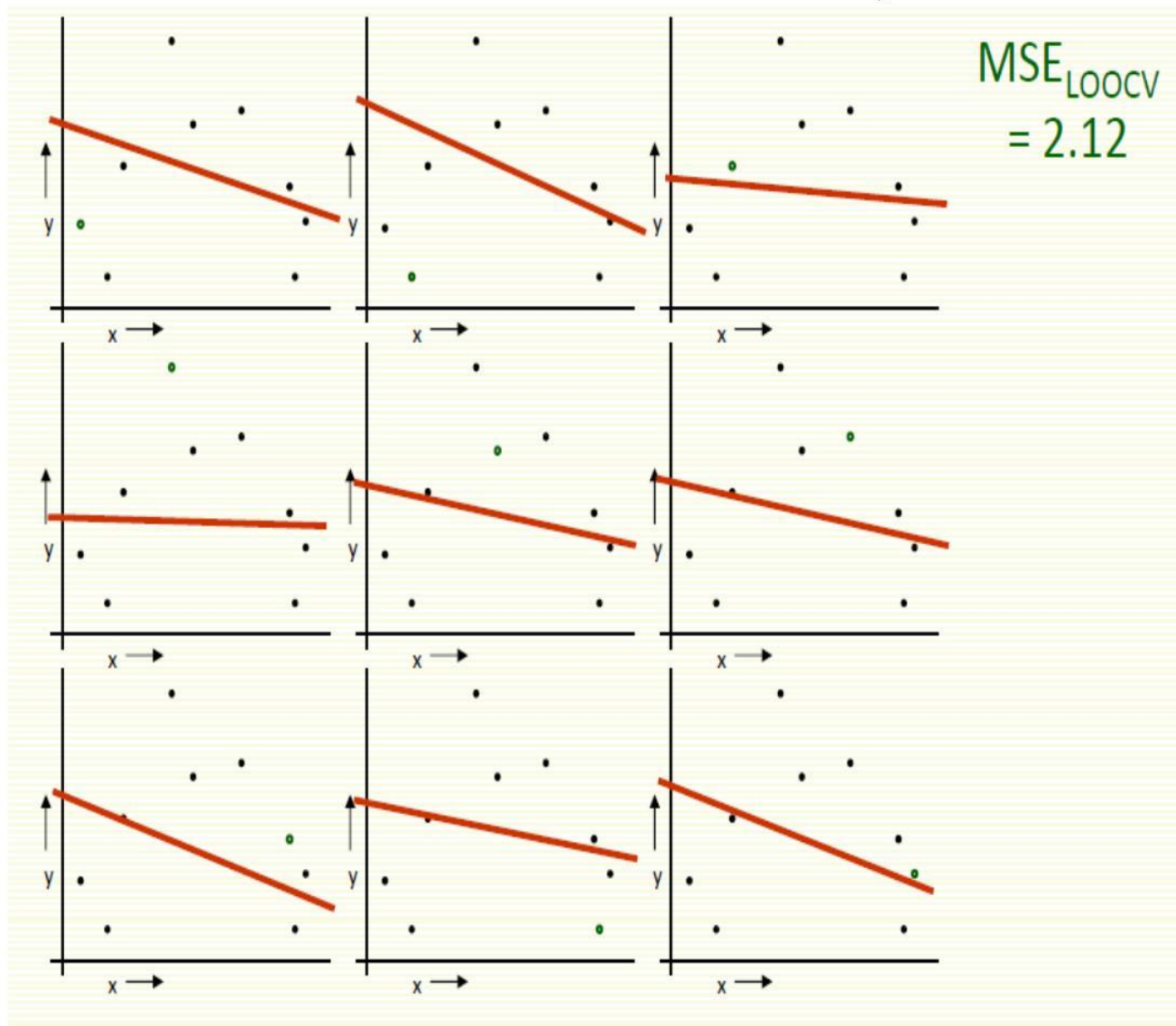


For $k=1$ to n

1. Let $(\mathbf{x}^k, \mathbf{y}^k)$ be the k th example
2. Temporarily remove $(\mathbf{x}^k, \mathbf{y}^k)$ from the dataset
3. Train on the remaining $n-1$ examples
4. Note your error on $(\mathbf{x}^k, \mathbf{y}^k)$

When you've done all points,
report the mean error

LOOCV (Leave-one-out Cross Validation)



How cross validation helps in regularization?

- Cross-validation is used to choose the best **regularization strength (λ)** for L1 and L2 models.

We know,

L1 (Lasso): adds $\lambda \sum |w_i|$

L2 (Ridge): adds $\lambda \sum w_i^2$

- λ controls **how strong the penalty is**

How do we choose λ ?

- Small $\lambda \rightarrow$ weak regularization $\rightarrow$ **overfitting**
- Large $\lambda \rightarrow$ strong regularization $\rightarrow$ **underfitting**
- We don't know the best λ beforehand.

Solution: Use Cross-Validation

Try multiple values of λ and test each using cross-validation.

Process of Cross Validation

- Choose values of λ (e.g., 0.01, 0.1, 1, 10)
- For each λ :
 - Train model (L1 or L2)
 - Perform **k-fold cross-validation**
 - Compute average validation error
- Compare results
- Select λ with **lowest validation error**

λ	CV Error
0.01	High (overfit)
0.1	Lower
1	Lowest
10	High (underfit)

Choose $\lambda = 1$

- Regularization **controls** model complexity
- Cross-validation **estimates** generalization performance
- Together:
 - They help find the best **balance between bias and variance**

Advantages of Cross Validation

- Better performance estimate
- Uses data efficiently
- Reduces overfitting risk

Limitation of Cross Validation

- Computationally expensive
- Slower for large datasets

Summary

Empirical risk minimization (ERM)	minimizes training error only.
Structural risk minimization (SRM)	minimizes training error plus a complexity penalty to improve generalization
Regularization	$L1/L2 \rightarrow$ control complexity $\lambda \rightarrow$ strength of regularization
Cross-validation	How we choose best model / λ